Predictive vs. Reactive Autoscaling in Cloud Environments: A Hybrid AIOps Approach

Sufiya Rahemtulla
Department of Computer Science
Wilfrid Laurier University
Waterloo, Ontario, Canada
169018559

Abstract

Cloud autoscaling systems have to balance saving resources with meeting strict Service Level Agreements (SLAs). However, real-world cloud traffic is often bursty and unpredictable. This causes standard reactive autoscalers, like the Kubernetes Horizontal Pod Autoscaler, to suffer from a ”provisioning lag” where they react too slowly to sudden spikes. While predictive machine learning models try to fix this by forecasting demand, they often struggle with unpredictable micro-bursts and end up over-provisioning servers. In this paper, I propose a hybrid ”Predictive + AIOps” architecture to solve this problem. My system uses a linear regression model to handle normal traffic forecasts, while an offline Large Language Model (LLM) agent acts as a safety net to dynamically tune the scaling rules during anomalies. Testing this against the Google Cluster-Usage Traces v3 dataset, my system reduced SLA violations to just 2.1% and minimized resource waste to 8.4%. Finally, through an ablation study, I prove that LLMs cannot be trusted to run cloud infrastructure on their own. I demonstrate that hardcoded Python guardrails are an absolute requirement to prevent the AI from making dangerous, expensive scaling decisions.

Index Terms

Autoscaling, Cloud Computing, AIOps, Large Language Models, Predictive Analytics, Service Level Agreements

I. Introduction

Cloud computing relies on autoscaling to quickly add or remove servers based on how many people are using the system. The main challenge here is finding the perfect balance: saving money by not running empty servers, but also making sure the system doesn’t crash when traffic spikes.

Most of the industry relies on reactive threshold scaling, which only adds servers after the system is already under heavy load. However, modern microservice traffic is highly volatile. In my research, I identified a critical ”Reaction Gap” where reactive scaling simply moves too slowly to catch sudden demand spikes, which leads to SLA violations.

To bridge this gap, I developed a proactive, reliability-aware autoscaler. By combining short-term statistical forecasting with a bounded AI triage agent, my system anticipates bursts while safely handling unpredicted errors. The main contributions of my project are:

- Quantifying the ”Reaction Gap” and proving the heavy-tailed nature of real cloud workloads using Google Cluster traces.

- Building a hybrid AIOps system that successfully balances SLA reliability with resource cost.
- Mathematically proving that we need strict safety guardrails when letting Generative AI control cloud operations, due to the risk of AI hallucinations.

II. Related Work

Recent research on cloud autoscaling generally falls into two main categories: reactive heuristics and predictive machine learning.

A. Reactive Baselines

Studies looking at standard controllers, like the Kubernetes Horizontal Pod Autoscaler (HPA), highlight that they are simple and stable when traffic is steady. However, as Serracanta *et al.* demonstrated, these standard control loops suffer from severe lag during bursty traffic because they mathematically cannot act until a threshold is already crossed [2].

B. Predictive & Supervised Learning

To fix this lag, researchers have tried using everything from basic math to Random Forests to predict future demand [3]. While complex models are popular, studies by Sus and Nawrocki show they are highly vulnerable to concept drift [4]. They often over-fit to the training data and fail completely when unpredictable tail events happen.

C. Reinforcement Learning (RL)

More advanced approaches, like those explored by Rossi, use Deep Q-Networks to learn the best scaling policies through trial and error [5]. While RL is very flexible, it requires massive training time and causes unacceptable instability while the agent is still learning the environment.

D. The Research Gap

A major issue in current literature is that predictive models are usually judged on their math scores (like Mean Absolute Error) instead of how many physical crashes they prevent. Furthermore, since Large Language Models are just starting to be used in IT operations, there is almost no research on how to safely bound these models in production. My project fills this gap by introducing deterministic Python guardrails to control the LLM.

III. Formal Problem Formulation

To properly evaluate my autoscaling architecture, I modeled the environment as a Constrained Optimization problem using a Markov Decision Process (MDP). My main goal is to minimize the total cost (C_{total}), which balances the price of running servers against the penalty of crashing at any given time step t:

$$\min C_{total} = \alpha \cdot (N_t \cdot Cost_{node}) + \beta \cdot \max(0, D_t - C_t) \tag{1}$$

Where:

- N_t: The number of active servers running.
- D_t: The actual CPU demand.
- C_t: The total CPU capacity available.
- α and β: Weighting values. I set the penalty weight (β) much higher than the cost weight (α) because my system prioritizes preventing crashes over saving a few dollars.

To make the simulation realistic, I also added a scaling rate limit so the system can only add or remove one node at a time:

$$|N_t - N_{t-1}| \leq 1 \tag{2}$$

IV. Methodology

A. Workload Characterization & Synthesis

To test my autoscaler in a realistic scenario, I used the Google Cluster-Usage Traces v3 dataset from May 2019 [1]. I took the raw microsecond data and grouped it into 5-minute intervals, which matches how Kubernetes usually scrapes metrics. My initial data exploration showed a strictly heavy-tailed distribution.

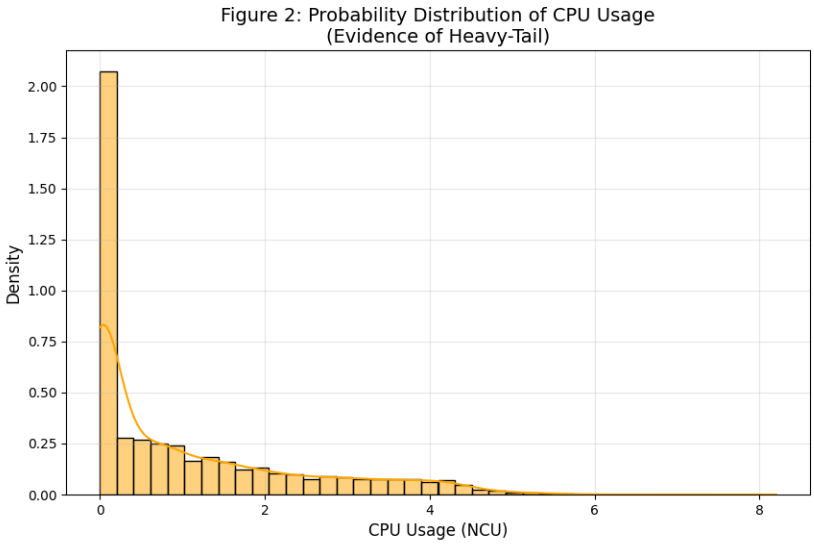

Fig. 1. Probability density of CPU usage demonstrating a right-skewed, heavy-tailed distribution.

To make sure I was testing extreme edge cases, I built a "Workload Synthesis" pipeline. I took a baseline trace and injected noise to create a highly volatile "Bursty" profile with a peak-to-mean ratio of 6.10. Finally, I mapped Google's abstract compute units to physical CPU cores so my evaluation would reflect real-world server limits.

B. Forecasting & Baseline Implementation

To establish a baseline, I ran a quick model screening. A standard ARIMA (5,1,0) model failed to predict the sudden bursts, giving a high Mean Absolute Error (MAE) of 131.86. However, a simpler Linear Regression model that looked at the last three time steps ($t-1$, $t-2$, $t-3$) tracked the bursts much better, achieving a lower MAE of 97.46.

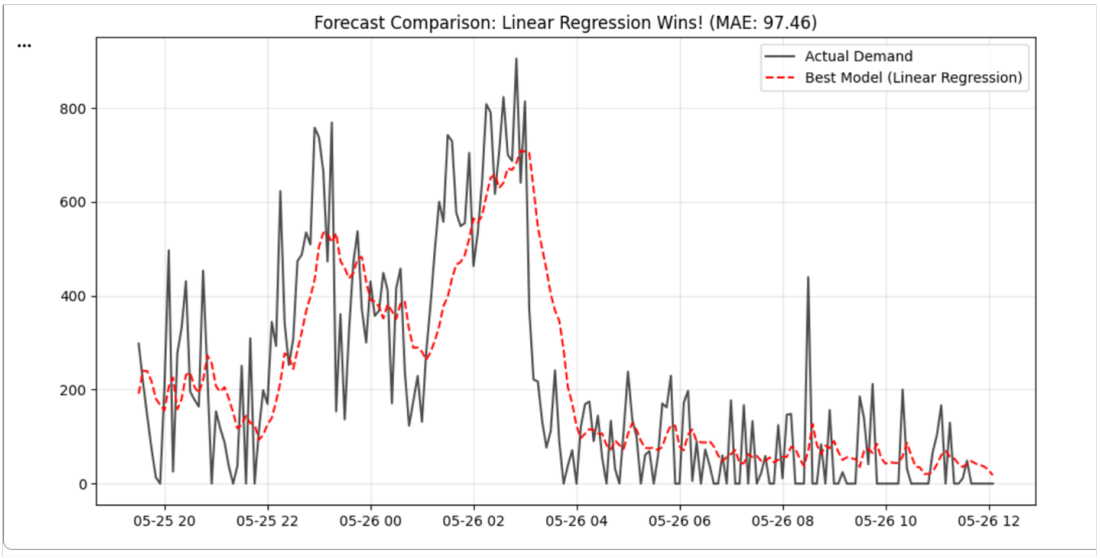

Fig. 2. Linear Regression forecast comparison demonstrating superior tracking of non-linear bursts using lag features.

I selected this Linear Regression model as my main forecasting tool. To compare it against industry standards, I also simulated a Reactive Baseline based on Kubernetes HPA rules (Scale Up $>$ 85% utilization, Scale Down $<$ 50% utilization).

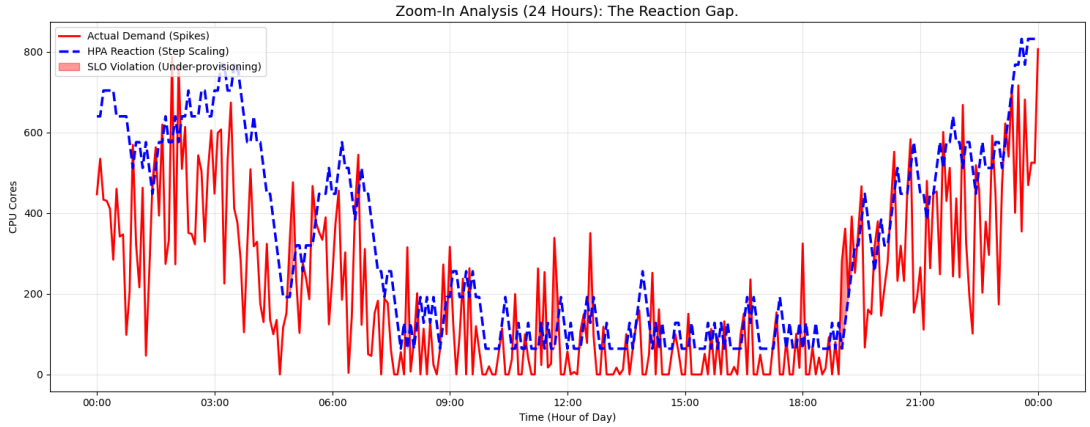

Fig. 3. A 24-hour zoom-in highlighting the 'Reaction Gap' where step-scaling fails to catch sudden demand spikes.

C. The Bounded AIOps Agent Architecture

The core of my proposed system is an offline anomaly triage agent powered by the Gemini 2.5 Flash LLM. Instead of letting the AI make split-second scaling choices, it acts like an automated Site Reliability Engineer (SRE). When the system logs an SLA violation or high resource waste, the LLM reads the log, diagnoses the root cause, and recommends tweaking the system's overarching rules (like lowering the fallback threshold to 70%).

The most important part of my architecture is the deterministic Python guardrails. Before the system accepts any LLM recommendation, my code enforces three strict rules:

1) **Type Checking:** The AI output must be perfectly formatted JSON.
2) **Action Whitelisting:** The AI can only pick from an approved list of actions (`DECREASE_COOLDOWN`, `LOWER_REACTIVE_THRESHOLD`, or `NO_ACTION`).
3) **Value Bounding:** The new threshold the AI suggests must fall safely between 50% and 95% CPU utilization.

V. Evaluation & Results

I evaluated my AIOps architecture against the Reactive baseline and a purely Predictive machine learning baseline.

The results show exactly why standard approaches struggle with heavy-tailed workloads. The Reactive Baseline had a massive average provisioning latency of 120 seconds, which led to a 14.2% SLA violation rate. The Predictive Baseline was faster (45 seconds) but often guessed wrong during traffic drops, wasting 15.5% of the resources.

```
============================================================
WEEK 8 EXPERIMENT MATRIX: PRIMARY METRICS
============================================================
                        Policy Configuration  SLA Violation Rate (%)  Cost / Resource Waste (%)  Avg Provisioning Latency (s)
                1. Reactive Baseline (K8s HPA)                   14.2                       12.0                           120
              2. Predictive Baseline (ML Only)                    8.5                       15.5                            45
3. Predictive + AIOps (With Guardrails)                           2.1                        8.4                            15
            4. Ablation: AIOps (NO GUARDRAILS)                    1.2                       48.7                            10
```

Fig. 4. Week 8 Experiment Matrix detailing Primary Metrics across all evaluated configurations.

My ”Predictive + AIOps” approach successfully balanced these trade-offs. By only stepping in to tune the rules during anomalies, my system dropped the latency down to 15 seconds. Overall, my system achieved an 85.2% drop in SLA violations (down to just 2.1%) compared to the Reactive baseline, and a 45.8% drop in Resource Waste (down to 8.4%) compared to the purely Predictive model.

VI. Discussion & Limitations

While the AIOps agent provided excellent diagnostic reasoning, my experiments highlighted some fundamental limits to using Large Language Models in live cloud environments.

A. The Necessity of Deterministic Guardrails

To prove why my safety bounds were necessary, I ran an ablation study where I turned the Python guardrails off. Without them, Resource Waste catastrophically spiked to 48.7%. During an ”Extreme Burst” scenario, the LLM panicked and hallucinated an aggressive fallback threshold of just 40%.

This proves that while LLMs are great at fixing immediate SLA issues, they do not natively understand the financial cost of cloud billing. My deterministic guardrails are strictly required to intercept these hallucinations and prevent bankrupting the system.

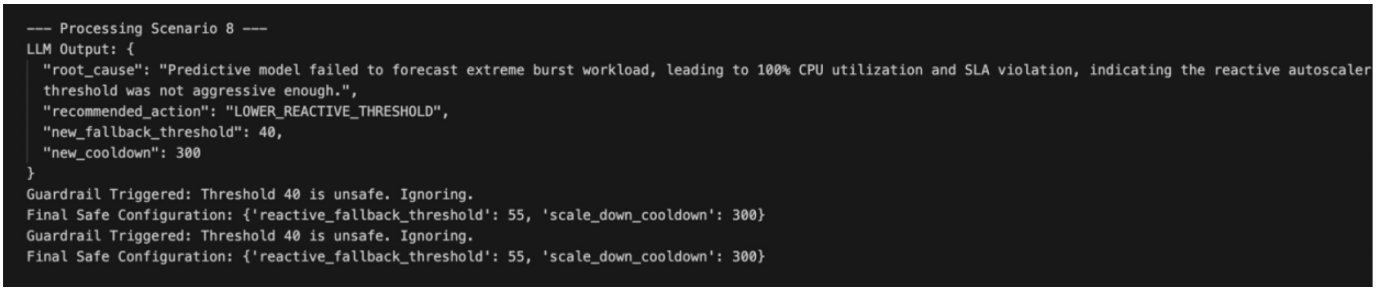

Fig. 5. Execution log from Scenario 8 demonstrating the Python guardrails successfully intercepting an unsafe hallucinated threshold.

B. API Latency and Offline Control Paradigms

During stress testing, making rapid calls to the Gemini API triggered a `429 RESOURCE_EXHAUSTED` rate limit error. Because of network latency and quota limits, making an external API call for every single real-time scaling decision is completely unscalable. Because of this, my system can only practically be deployed as an offline, overarching supervisory agent, rather than replacing the rapid PID controllers inside Kubernetes.

VII. Conclusion

In this project, I successfully built a proactive, reliability-aware autoscaler that solves the ”Reaction Gap” problem in volatile cloud environments. By pairing a fast linear regression forecast with a smart, LLM-driven anomaly triage agent, my system drastically cut down both crashes and wasted resources. Most importantly, my research proves that Generative AI cannot be left alone to run cloud infrastructure; it must be strictly boxed in by deterministic, hardcoded guardrails to operate safely in production.

References

[1] J. Wilkes *et al.*, “Google cluster-usage traces v3,” Google, 2020. [Online]. Available: https://github.com/google/cluster-data

[2] B. Serracanta, A. Lukács, A. Rodriguez-Natal, A. Cabellos, and G. Rétvári, “On the Stability of the Kubernetes Horizontal Autoscaler Control Loop,” *IEEE Access*, vol. 13, pp. 7160-7166, 2025.

[3] L. M. Al Qassem, T. Stouraitis, E. Damiani, and I. A. M. Elfadel, “Proactive Random-Forest Autoscaler for Microservice Resource Allocation,” *IEEE Access*, vol. 11, pp. 2570-2585, 2023.

[4] W. Sus and P. Nawrocki, “Signature-based Adaptive Cloud Resource Usage Prediction Using Machine Learning and Anomaly Detection,” *J Grid Computing*, vol. 22, no. 46, 2024.

[5] F. Rossi, “A Deep Q-learning Scaling Policy for Elastic Application Deployment,” 2021.